ELITE BEGINNER CHALLENGE

Four-Concept Test | Python • Recursion • Big O • Hash Maps • Decorators

Purpose

This assessment checks whether you can read, reason about, write, test, and explain Python—not merely repeat definitions. You will work with real Python behavior, trace execution, implement functions, and justify design decisions.

Assessment rules

- Language: Python 3. No external libraries.
- Suggested time: 90–120 minutes. Total: 100 marks.
- Try each question before searching for help. You may use Python to test your own predictions only after writing them down.
- For every coding task, include at least one normal case and one edge case.
- Explain your reasoning in plain English. Correct code with no understanding earns partial credit.
- Do not use a built-in shortcut when the question explicitly forbids it.

QUEST 1 — Recursion: Follow the Calls (20 marks)

Learning target: Understand base cases, recursive cases, call stacks, return values, and progress toward termination.

A student writes this function to add the integers from 1 through n:

```
def total(n):
    if n == 1:
        return 1
    return n + total(n - 1)
```

A. Trace it (5 marks): Without running the code, trace total(4). List the calls in the order they begin, then show how the return values travel back.

B. Find the problem (5 marks): What happens for total(0) and total(-3)? Rewrite the function so negative inputs raise ValueError and zero returns 0. State the behavior you chose for non-integer input.

C. Explain termination (5 marks): In your own words, distinguish the base case from the recursive case. Explain why each call must move toward a stopping condition. What failure occurs if the stopping condition is never reached?

D. Code it (5 marks): Write countdown_sum(n), which recursively returns the sum of odd positive integers from n down to 1. Return 0 for n <= 0. Do not use loops, sum(), count(), or comprehensions.

Examples:

```
countdown_sum(5)  # 9: 5 + 3 + 1
countdown_sum(4)  # 4: 3 + 1
countdown_sum(1)  # 1
countdown_sum(0)  # 0
```

Python learning note: A recursive call is an ordinary function call. Each call has its own argument values and waits for the next call to return.

QUEST 2 — Big O: Growth, Not Guesswork (25 marks)

Learning target: Connect Python operations and loops to time complexity, auxiliary space, and practical trade-offs.

Each function checks whether a list contains a duplicate:

```
def has_duplicate_a(items):
    for i in range(len(items)):
        for j in range(i + 1, len(items)):
            if items[i] == items[j]:
                return True
    return False

def has_duplicate_b(items):
    seen = set()
    for item in items:
        if item in seen:
            return True
        seen.add(item)
    return False

def has_duplicate_c(items):
    items = sorted(items)
    for i in range(1, len(items)):
        if items[i] == items[i - 1]:
            return True
    return False
```

A. Analyze (9 marks): For each implementation, state the worst-case or expected time complexity as appropriate, plus auxiliary space complexity. Point to the operations that create that cost.

B. Predict before running (5 marks): Compare a list of 10,000 distinct integers with a list whose first two values are equal. Which implementations can return early? Which is likely to do the most work on distinct values? Explain why Big O is not an exact stopwatch.

C. Correct the misconception (5 marks): A student says, “Implementation B is O(1) because set membership is O(1).” Explain the difference between the expected cost of one set lookup and the cost of processing n items.

D. Make an engineering choice (6 marks): Choose an implementation for millions of records. Discuss speed, memory, sorting/input mutation, and what collision-heavy hashing could mean. A strong answer names a trade-off.

Use this reminder: Big O describes how resource use grows as input size grows; it does not promise an exact number of seconds.

Optional self-check after completing your written analysis: test n = 10, 100, and 1,000 with both duplicate and no-duplicate cases. Record what changes and what does not.

QUEST 3 — Hashing and Hash Maps: Build the Mechanism (25 marks)

Learning target: Understand keys, hash indexes, collisions, buckets, and why a hash map still needs to compare keys.

You may use lists, tuples, loops, and conditionals, but you may NOT use dict or set in this question.

```
def hash_key(key):
    return sum(ord(char) for char in key) % 7
```

A. Calculate (5 marks): Manually calculate the slot index for 'cat', 'act', 'dog', and 'god'. Show the character-code sum and remainder for each.

B. Reason about collisions (5 marks): Identify any collisions. Explain why different strings can map to the same index and why a matching index does not prove two keys are equal.

C. Implement (10 marks): Build a seven-slot hash map using separate chaining. Implement put(key, value), get(key), and delete(key). put must update an existing key rather than duplicate it; get must return None for a missing key; delete must remove only the requested key.

D. Analyze (5 marks): Explain expected time complexity under reasonable hashing assumptions and worst-case complexity if all keys land in one bucket. Explain how a larger table may help without guaranteeing O(1) for every operation.

Implementation guidance: Each slot can hold a list of key-value pairs. To retrieve a value, compute the bucket index, then scan that bucket and compare the stored key with the requested key.

Do not confuse the hash function with the whole hash map. The hash function chooses a bucket; collision handling and key comparison are still necessary.

QUEST 4 — Decorators: Wrap Behavior Without Breaking It (20 marks)

Learning target: Understand function objects, decoration time, wrapper calls, arguments, and return values.

```
def announce(func):
    print("Decorator is running")

    def wrapper(*args, **kwargs):
        print("Before function")
        result = func(*args, **kwargs)
        print("After function")
        return result

    return wrapper

@announce
def multiply(a, b):
    print("Multiplying")
    return a * b

print("Program started")
answer = multiply(3, 4)
print("Answer:", answer)
```

A. Predict output (5 marks): Write every printed line in exact order before running the program. Explain why the decorator's first print happens before “Program started.”

B. Explain the parts (5 marks): What does @announce do? What is wrapper? Why are *args and **kwargs useful? Why does returning result matter?

C. Repair a bug (5 marks): If wrapper calls func(*args, **kwargs) but does not return its result, answer becomes None. Explain why and correct the wrapper.

D. Implement (5 marks): Write count_calls, a decorator that tracks how many times a decorated function is called. The count must persist, the original result must be returned, and positional arguments must work. Demonstrate three calls and show the final count.

Python learning note: A decorator usually receives a function and returns another callable. The wrapper can add behavior before or after calling the original function.

QUEST 5 — Reflection and Debugging (10 marks)

Choose the concept you found hardest. Write a short explanation of your first incorrect assumption, the evidence that changed your mind, and one test that would catch the same mistake in future.

Required evidence: Include one edge case, one predicted output, and one sentence explaining why your solution is correct.

Marking guide

Assessment area	Marks
Recursion and tracing	20
Big O reasoning	25
Hash maps and collisions	25
Decorators	20
Reflection and debugging	10
TOTAL	100

Suggested interpretation: 90–100 = strong understanding for this stage; 75–89 = good foundation with gaps; 60–74 = can apply some patterns but needs practice; below 60 = revisit the concepts and trace more code. These are teaching guidelines, not standardized scores.

Learner review notes

The concept I understand best is…

_____

The concept I need to revisit is…

_____

One bug I found and fixed was…

_____

One question I can now answer without memorizing is…

_____